Computer Organization and Architecture Laboratory

MiniRISC Assignment 6: Autonomous Program Execution from Instruction BRAM

Indian Institute of Technology Kharagpur
Department of Computer Science and Engineering

Laboratory Date: 2 November 2026

Objective

Extend the MiniRISC processor developed up to 28 October so that it executes a complete program **autonomously from the instruction BRAM**.

After reset is released, the processor must begin execution from the initial program address, fetch instructions from the instruction BRAM, execute them without manual instruction entry, and stop when the program reaches `HALT`.

The required program computes the sum of four 32-bit values stored in data BRAM:

$$10 + 20 + 30 + 40 = 100$$

and stores the final result into data-memory word address 8.

The processor must then enter the halted state and assert `done`.

1. Required Top-Level Interface

Keep the processor top-level interface small. The mandatory interface is:

```
module minirisc32_top (
    input  wire        clk,
    input  wire        reset,
    output wire        done,
    output wire [31:0] result_out
);
```

Signal	Width	Meaning
`clk`	1	Processor clock
`reset`	1	Active-high processor reset
`done`	1	Asserted when the program reaches `HALT`
`result_out`	32	Final value produced by the program

No manual instruction-entry input is required. No separate `start` signal is required. After reset is de-asserted, the processor should begin program execution automatically.

2. Required Autonomous Behavior

The processor must:

1. initialize the control state and PC while `reset`=1;
2. begin instruction fetch automatically after reset is released;
3. fetch instructions from instruction BRAM;
4. execute the program without external instruction control;
5. perform register, ALU, branch, load/store, and memory operations automatically;
6. enter the halted state when `HALT` is executed;
7. assert `done`=1;
8. hold the final result stable on `result_out`.

After `done`=1, the processor must remain halted until reset.

3. Instruction BRAM

Use the BRAM-based instruction ROM already integrated in the earlier MiniRISC assignments. The complete program for this assignment must be stored in this instruction BRAM.

The instruction BRAM must be initialized through a `.coe` file.

Because each group may have chosen a different ISA encoding, each group must convert the symbolic program given below into its own 32-bit machine instructions.

The instruction memory is **word addressed**. Under the common convention:

$$PC_{next} = PC + 1$$

for normal sequential execution.

4. Data BRAM Initialization

Initialize the data BRAM with:

Word Address	Initial Value	Purpose
0	10	Input 0
1	20	Input 1
2	30	Input 2
3	40	Input 3
8	0	Result location

Other locations may be initialized to zero.

The expected final memory value is:

$$\boxed{Mem[8] = 100 = 32'h00000064}.$$

5. Required Program

The symbolic MiniRISC program is:

```
        ADDI R1, R0, 0      ; R1 = current data-memory address
        ADDI R2, R0, 4      ; R2 = number of elements
        ADDI R3, R0, 0      ; R3 = accumulator

LOOP:
        LD   R4, 0(R1)      ; R4 = Mem[R1]
        ADD  R3, R3, R4     ; R3 = R3 + R4

        ADDI R1, R1, 1      ; next word address
        SUBI R2, R2, 1      ; decrement count

        BZ   R2, DONE       ; exit when count becomes zero
        BR   LOOP            ; otherwise repeat

DONE:
        ST   R3, 8(R0)      ; Mem[8] = 100
        HALT
```

If a group's ISA does not contain exactly `SUBI` or `BR`, equivalent instructions from that group's approved ISA may be used.

The computation itself must remain unchanged.

6. Expected Register Behavior

A convenient register usage is:

Register	Purpose
$R0$	Constant zero
$R1$	Current data-memory word address
$R2$	Loop counter
$R3$	Accumulator
$R4$	Loaded data value

The accumulator should progress as:

$$0 \rightarrow 10 \rightarrow 30 \rightarrow 60 \rightarrow 100.$$

At program completion:

$$R3 = 100$$

and

$$Mem[8] = 100.$$

7. Suggested Instruction BRAM Locations

Under a simple word-addressed instruction-memory convention:

Instruction Address	Symbolic Instruction
0	`ADDI R1, R0, 0`
1	`ADDI R2, R0, 4`
2	`ADDI R3, R0, 0`
3	`LD R4, 0(R1)`
4	`ADD R3, R3, R4`
5	`ADDI R1, R1, 1`
6	`SUBI R2, R2, 1`
7	`BZ R2, DONE`
8	`BR LOOP`
9	`ST R3, 8(R0)`
10	`HALT`

The actual hexadecimal instruction values depend on the group's ISA encoding. Branch offsets must follow the branch convention already used by that group.

8. HALT and done Behavior

When `HALT` is executed:

- the processor must enter the halted state;
- the PC must stop changing;
- register-file write enable must remain inactive;
- data-memory write enable must remain inactive;
- `done` must become 1.

The halted state must persist until reset.

9. result_out

The final output must be:

$$\boxed{\texttt{result_out} = 32'h00000064}.$$

The value must originate from the actual result generated by the processor. It must not be hard-coded.

A convenient implementation is to capture the value stored into the designated result location when the final store completes, or to expose the actual value read from that result location after program completion.

10. Behavioral Simulation

Before implementation, verify in behavioral simulation that:

1. the PC starts from the chosen reset address;
2. the program is fetched from instruction BRAM;
3. the loop executes four times;
4. the four data words are loaded correctly;
5. the accumulator reaches 100;
6. 100 is stored into data-memory word address 8;
7. `HALT` stops execution;
8. `done`=1;
9. `result_out`=32'h00000064.

11. Post-Implementation Timing Simulation

The final verification for this assignment must include a **post-implementation timing simulation**.

Students must:

1. synthesize the complete MiniRISC design;
2. implement the design through placement and routing;
3. generate the post-implementation timing simulation model;
4. run the implemented netlist with timing delays;
5. verify complete autonomous program execution;
6. verify `done` and `result_out` after `HALT`.

Bitstream generation is not required.

12. Clock Requirement

Use the normal MiniRISC processor clock selected by the group and constrain it correctly during implementation.

For example, for a 100 MHz processor clock:

$$T_{clk} = 10 \text{ ns}.$$

The timing-simulation testbench must use the same clock period.

13. Minimal Timing-Simulation Testbench

Because the processor executes autonomously, the testbench can remain small.

```
module tb_minirisc_autonomous;

reg clk;
reg reset;

wire done;
wire [31:0] result_out;

minirisc32_top dut (
    .clk        (clk),
    .reset      (reset),
    .done       (done),
    .result_out (result_out)
);

initial
    clk = 1'b0;

always #5 clk = ~clk;     // Example: 100 MHz

initial begin
    reset = 1'b1;

    #50;
    reset = 1'b0;

    wait(done == 1'b1);

    if (result_out == 32'h00000064)
        $display("PASS: result = %h", result_out);
    else
        $display("FAIL: result = %h", result_out);

    // Keep running briefly to verify stable HALT behavior
    #100;
    $finish;
end

endmodule
```

Reset duration and total simulation duration may be adjusted according to the implementation.

14. Signals for Demonstration

The mandatory top-level signals are only:

- `clk`;
- `reset`;
- `done`;
- `result_out[31:0]`.

For debugging and behavioral verification, students may also inspect internal signals such as PC, current instruction, FSM state, accumulator value, BRAM address, and BRAM write enable.

These internal signals do **not** need to be exposed as permanent top-level ports.

15. Required Final Timing-Simulation Result

The post-implementation timing simulation must end with:

$$\boxed{\texttt{done} = 1}$$

and

$$\boxed{\texttt{result_out} = 32'h00000064}.$$

Both values must remain stable after the processor has halted.

16. TA Demonstration

During the laboratory demonstration, show:

1. instruction BRAM initialized with the encoded program;
2. data BRAM initialized with 10, 20, 30, and 40;
3. successful behavioral simulation;
4. successful post-implementation timing simulation;
5. final `result_out`=32'h00000064;
6. `done`=1 after `HALT`;
7. stable halted behavior after completion.

17. Files to Submit

Submit:

- complete MiniRISC Verilog source files;
- instruction BRAM `.coe` file;
- data BRAM `.coe` file;
- behavioral testbench;
- post-implementation timing-simulation testbench;
- behavioral simulation waveform;
- post-implementation timing-simulation waveform.

No FPGA-board demonstration or bitstream generation is required for this assignment.